

LAB – 2

Aim : To perform exploratory data analysis (EDA) on a student performance dataset using Pandas and Matplotlib.

Dataset

Create a CSV file named student_data.csv with the following data:

RollNo	Name	Gender	Attendance	InternalMarks	ExternalMarks
101	Ravi	M	85	32	55
102	Priya	F	92	38	58
103	Kiran	M	78	28	45
104	Anu	F	95	40	60
105	Sita	F	88	35	54
106	Raj	M	70	25	40
107	Divya	F	90	37	57
108	Arun	M	82	30	48
109	Keerthi	F	96	39	59
110	Naveen	M	75	27	44

Tasks

Task A: Dataset Loading and Inspection

- Import the necessary Python libraries.
- Load the dataset into a Pandas DataFrame.
- Display the first five records.
- Display the last five records.
- Find the total number of rows and columns.
- Display the names of all columns.
- Display the data type of each column.
- Generate dataset information using info().

Code:

```
#Task A
import pandas as pd

from google.colab import files

uploaded = files.upload()
```

```

df = pd.read_csv("student_data.csv")

print(df)

# Display the first five records
print(df.head(5))

#Display the last five records
print(df.tail(5))

# Find the total number of rows and columns

print(df.shape[0])
print(df.shape[1])

#Display the names of all columns.
print(df.columns.tolist())

# Display the datatype of each column
print(df["RollNo"].dtype)
print(df["Name"].dtype)
print(df["Gender"].dtype)
print(df["Attendance"].dtype)
print(df["InternalMarks"].dtype)
print(df["ExternalMarks"].dtype)

#Generate dataset information using info()
print(df.info())

```

OUTPUT:

... Choose Files student_data.csv

student_data.csv(text/csv) - 275 bytes, last modified: 6/27/2026 - 100% done

Saving student_data.csv to student_data.csv

	RollNo	Name	Gender	Attendance	InternalMarks	ExternalMarks
0	101	Ravi	M	85	32	55
1	102	Priya	F	92	38	58
2	103	Kiran	M	78	28	45
3	104	Anu	F	95	40	60
4	105	Sita	F	88	35	54
5	106	Raj	M	70	25	40
6	107	Divya	F	90	37	57
7	108	Arun	M	82	30	48
8	109	Keerthi	F	96	39	59
9	110	Naveen	M	75	27	44

```
# Display the first five records
print(df.head(5))
```

	RollNo	Name	Gender	Attendance	InternalMarks	ExternalMarks
0	101	Ravi	M	85	32	55
1	102	Priya	F	92	38	58
2	103	Kiran	M	78	28	45
3	104	Anu	F	95	40	60
4	105	Sita	F	88	35	54

```
#Display the last five records
print(df.tail(5))
```

...	RollNo	Name	Gender	Attendance	InternalMarks	ExternalMarks
5	106	Raj	M	70	25	40
6	107	Divya	F	90	37	57
7	108	Arun	M	82	30	48
8	109	Keerthi	F	96	39	59
9	110	Naveen	M	75	27	44

```
# Find the total number of rows and columns
```

```
print(df.shape[0])
print(df.shape[1])
```

```
10
6
```

```
#Display the names of all columns.
print(df.columns.tolist())
```

```
['RollNo', 'Name', 'Gender', 'Attendance', 'InternalMarks', 'ExternalMarks']
```



```
# Display the datatype of each column
print(df["RollNo"].dtype)
print(df["Name"].dtype)
print(df["Gender"].dtype)
print(df["Attendance"].dtype)
print(df["InternalMarks"].dtype)
print(df["ExternalMarks"].dtype)
```

```
int64
object
object
int64
int64
int64
```

```
▶ #Generate dataset information using info()
print(df.info())
```

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   RollNo          10 non-null    int64
1   Name            10 non-null    object
2   Gender          10 non-null    object
3   Attendance      10 non-null    int64
4   InternalMarks   10 non-null    int64
5   ExternalMarks   10 non-null    int64
dtypes: int64(4), object(2)
memory usage: 612.0+ bytes
None
```

Task B: Statistical Analysis

- Calculate the mean attendance of all students.
- Calculate the average Internal Marks.
- Calculate the average External Marks.
- Find the maximum Internal Marks.
- Find the minimum Internal Marks.
- Find the maximum External Marks.
- Find the minimum External Marks.
- Calculate the standard deviation of Internal Marks.
- Generate summary statistics using describe().

Code:

```
# Task B
```

```
#Calculate the mean attendance of all students.
```

```
print(df["Attendance"].mean())
```

```
#Calculate the average Internal Marks
```

```
print(df["InternalMarks"].mean())
```

```
#Calculate the average External Marks
```

```
print(df["ExternalMarks"].mean())
```

```
#Find the maximum Internal Marks.
```

```
print(df["InternalMarks"].max())
```

```
#Find the minimum Internal Marks.
```

```
print(df["InternalMarks"].min())
```

```
#Find the maximum External Marks.
```

```
print(df["ExternalMarks"].max())
```

```
#Find the minimum External Marks.
```

```
print(df["ExternalMarks"].min())
```

```
# Calculate the standard deviation of Internal Marks.
```

```
print(df["InternalMarks"].std())
```

```
#Generate summary statistics using describe().
```

```
print(df.describe())
```

OUTPUT:

```
85.1
```

```
33.1
```

52.0				
40				
25				
60				
40				
5.425249610233001				
...	RollNo	Attendance	InternalMarks	ExternalMarks
count	10.00000	10.000000	10.00000	10.000000
mean	105.50000	85.100000	33.10000	52.000000
std	3.02765	8.736259	5.42525	7.149204
min	101.00000	70.000000	25.00000	40.000000
25%	103.25000	79.000000	28.50000	45.750000
50%	105.50000	86.500000	33.50000	54.500000
75%	107.75000	91.500000	37.75000	57.750000
max	110.00000	96.000000	40.00000	60.000000

Task C: Data Manipulation

- Create a new column named TotalMarks.
- TotalMarks=InternalMarks+ExternalMarks
- Create a new column named Percentage. 1. Percentage=TotalMarks/100× 100
- Display the updated DataFrame

Code:

#Task C

#Create a new column named TotalMarks.

df["TotalMarks"] = df["InternalMarks"] + df["ExternalMarks"]

print(df["TotalMarks"])

#Create a new column named Percentage.

Percentage=TotalMarks/100× 100


```
df["Percentage"] = df["TotalMarks"]/100*100
```

```
print(df["Percentage"])
```

```
#Display the updated DataFrame
```

```
print(df)
```

OUTPUT:

```
...  0      87
     1      96
     2      73
     3     100
     4      89
     5      65
     6      94
     7      78
     8      98
     9      71
     Name: TotalMarks, dtype: int64
```

```
0      87.0
1      96.0
2      73.0
3     100.0
4      89.0
5      65.0
6      94.0
7      78.0
8      98.0
9      71.0
     Name: Percentage, dtype: float64
```

...	RollNo	Name	Gender	Attendance	InternalMarks	ExternalMarks	\
0	101	Ravi	M	85	32	55	
1	102	Priya	F	92	38	58	
2	103	Kiran	M	78	28	45	
3	104	Anu	F	95	40	60	
4	105	Sita	F	88	35	54	
5	106	Raj	M	70	25	40	
6	107	Divya	F	90	37	57	
7	108	Arun	M	82	30	48	
8	109	Keerthi	F	96	39	59	
9	110	Naveen	M	75	27	44	
	TotalMarks	Percentage					
0	87	87.0					
1	96	96.0					
2	73	73.0					
3	100	100.0					
4	89	89.0					
5	65	65.0					
6	94	94.0					
7	78	78.0					
8	98	98.0					
9	71	71.0					

Task D: Filtering and Searching

- Display the details of students whose attendance is greater than 90%.
- Display the details of students whose TotalMarks exceed 90.
- Display the details of female students.
- Display the details of male students.
- Display students whose ExternalMarks are less than 50.

Code:

#Display the details of students whose attendance is greater than 90%.

```
df[df["Attendance"] > 90 ]
```

#Display the details of students whose TotalMarks exceed 90.

```
df[df["TotalMarks"] > 90]
```

#Display the details of female students.

```
df[df["Gender"] == "F"]
```

#Display the details of male students.

```
df[df["Gender"] == "M"]
```

Display students whose ExternalMarks are less than 50.

```
df[df["ExternalMarks"] < 50]
```


OUTPUT:

...

	RollNo	Name	Gender	Attendance	InternalMarks	ExternalMarks	TotalMarks	Percentage
1	102	Priya	F	92	38	58	96	96.0
3	104	Anu	F	95	40	60	100	100.0
8	109	Keerthi	F	96	39	59	98	98.0

...

	RollNo	Name	Gender	Attendance	InternalMarks	ExternalMarks	TotalMarks	Percentage
1	102	Priya	F	92	38	58	96	96.0
3	104	Anu	F	95	40	60	100	100.0
6	107	Divya	F	90	37	57	94	94.0
8	109	Keerthi	F	96	39	59	98	98.0

...

	RollNo	Name	Gender	Attendance	InternalMarks	ExternalMarks	TotalMarks	Percentage
1	102	Priya	F	92	38	58	96	96.0
3	104	Anu	F	95	40	60	100	100.0
4	105	Sita	F	88	35	54	89	89.0
6	107	Divya	F	90	37	57	94	94.0
8	109	Keerthi	F	96	39	59	98	98.0

	RollNo	Name	Gender	Attendance	InternalMarks	ExternalMarks	TotalMarks	Percentage
0	101	Ravi	M	85	32	55	87	87.0
2	103	Kiran	M	78	28	45	73	73.0
5	106	Raj	M	70	25	40	65	65.0
7	108	Arun	M	82	30	48	78	78.0
9	110	Naveen	M	75	27	44	71	71.0

...

	RollNo	Name	Gender	Attendance	InternalMarks	ExternalMarks	TotalMarks	Percentage
2	103	Kiran	M	78	28	45	73	73.0
5	106	Raj	M	70	25	40	65	65.0
7	108	Arun	M	82	30	48	78	78.0
9	110	Naveen	M	75	27	44	71	71.0

Task E: Ranking and Comparison

- Identify the student with the highest TotalMarks.
- Identify the student with the lowest TotalMarks.
- Display the top three performers based on TotalMarks.
- Display the bottom three performers based on TotalMarks.
- Determine how many students scored above the class average.

Code:

#Task E

```
# Highest TotalMarks
highest_student = df.nlargest(1, 'TotalMarks')['Name'].values[0]
print("Highest TotalMarks Student:", highest_student)

# Lowest TotalMarks
lowest_student = df.nsmallest(1, 'TotalMarks')['Name'].values[0]
print("Lowest TotalMarks Student:", lowest_student)

# Top 3 performers
print("\nTop 3 Performers:")
print(df.nlargest(3, 'TotalMarks')[['Name', 'TotalMarks']])

# Bottom 3 performers
print("\nBottom 3 Performers:")
print(df.nsmallest(3, 'TotalMarks')[['Name', 'TotalMarks']])

# Students above class average
avg = df['TotalMarks'].mean()
above_avg = df[df['TotalMarks'] > avg]
print(f"\nClass Average: {avg}")
print(f"Students above average: {len(above_avg)}")
print(above_avg[['Name', 'TotalMarks']])
```

OUTPUT:

```
... Highest TotalMarks Student: Anu
    Lowest TotalMarks Student: Raj
```

```
Top 3 Performers:
```

	Name	TotalMarks
3	Anu	100
8	Keerthi	98
1	Priya	96

```
Bottom 3 Performers:
```

	Name	TotalMarks
5	Raj	65
9	Naveen	71
2	Kiran	73

```
Class Average: 85.1
```

```
Students above average: 6
```

	Name	TotalMarks
0	Ravi	87
1	Priya	96
3	Anu	100
4	Sita	89
6	Divya	94
8	Keerthi	98

Task F: Group-wise Analysis

- Calculate the average TotalMarks of male students.
- Calculate the average TotalMarks of female students.
- Compare the average attendance of male and female students.

Code:

```
#Task F
```

```
# Average TotalMarks by Gender
```

```
avg_male = df[df['Gender'] == 'M']['TotalMarks'].mean()
```

```
avg_female = df[df['Gender'] == 'F']['TotalMarks'].mean()
```

```
print(f"Average TotalMarks - Male : {avg_male}")
```

```
print(f"Average TotalMarks - Female: {avg_female}")
```

```
# Average Attendance by Gender
```

```
att_male = df[df['Gender'] == 'M']['Attendance'].mean()
```

```
att_female = df[df['Gender'] == 'F']['Attendance'].mean()
```



```
print(f"\nAverage Attendance - Male : {att_male}")
print(f"Average Attendance - Female: {att_female}")
```

OUTPUT:

```
*** Average TotalMarks - Male   : 74.8
    Average TotalMarks - Female: 95.4

    Average Attendance - Male   : 78.0
    Average Attendance - Female: 92.2
```

Task G: Data Visualization

- Plot a bar chart showing TotalMarks of all students.
- Plot a histogram of Attendance.
- Plot a pie chart showing the distribution of male and female students.
- Plot a scatter plot between Attendance and TotalMarks.
- Interpret the relationship between Attendance and TotalMarks.

Code:

#Task G

1. Bar Chart - TotalMarks of all students

```
plt.figure(figsize=(10, 5))
plt.bar(df['Name'], df['TotalMarks'], color='steelblue')
plt.title('TotalMarks of All Students')
plt.xlabel('Student Name')
plt.ylabel('TotalMarks')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

2. Histogram - Attendance

```
plt.figure(figsize=(7, 4))
plt.hist(df['Attendance'], bins=5, color='orange', edgecolor='black')
plt.title('Histogram of Attendance')
plt.xlabel('Attendance (%)')
plt.ylabel('Number of Students')
plt.tight_layout()
plt.show()
```

3. Pie Chart - Gender Distribution

```

gender_counts = df['Gender'].value_counts()
plt.figure(figsize=(5, 5))
plt.pie(gender_counts, labels=['Female', 'Male'], autopct='%1.1f%%',
        colors=['pink', 'skyblue'], startangle=90)
plt.title('Gender Distribution')
plt.show()

```

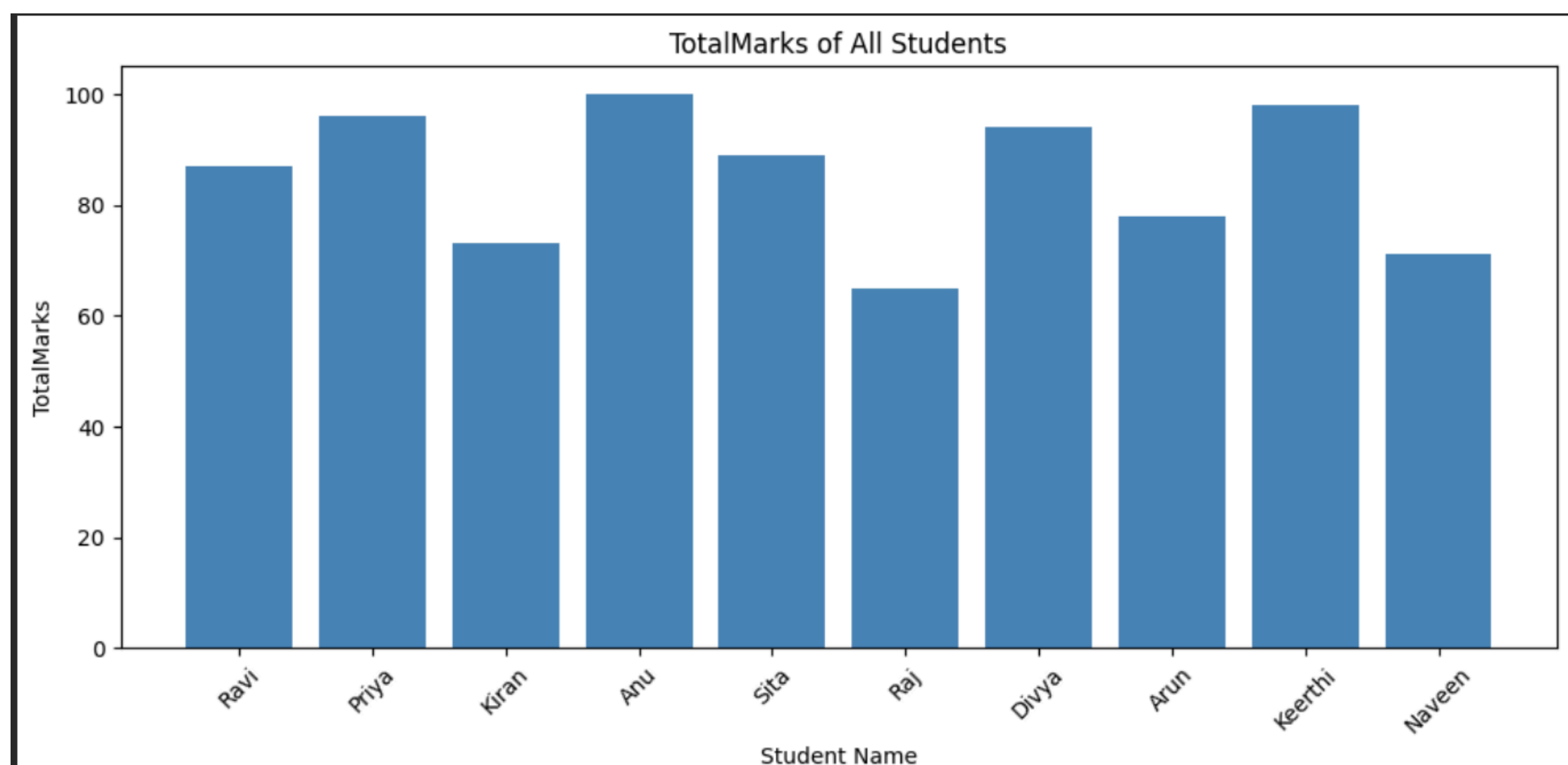
4. Scatter Plot - Attendance vs TotalMarks

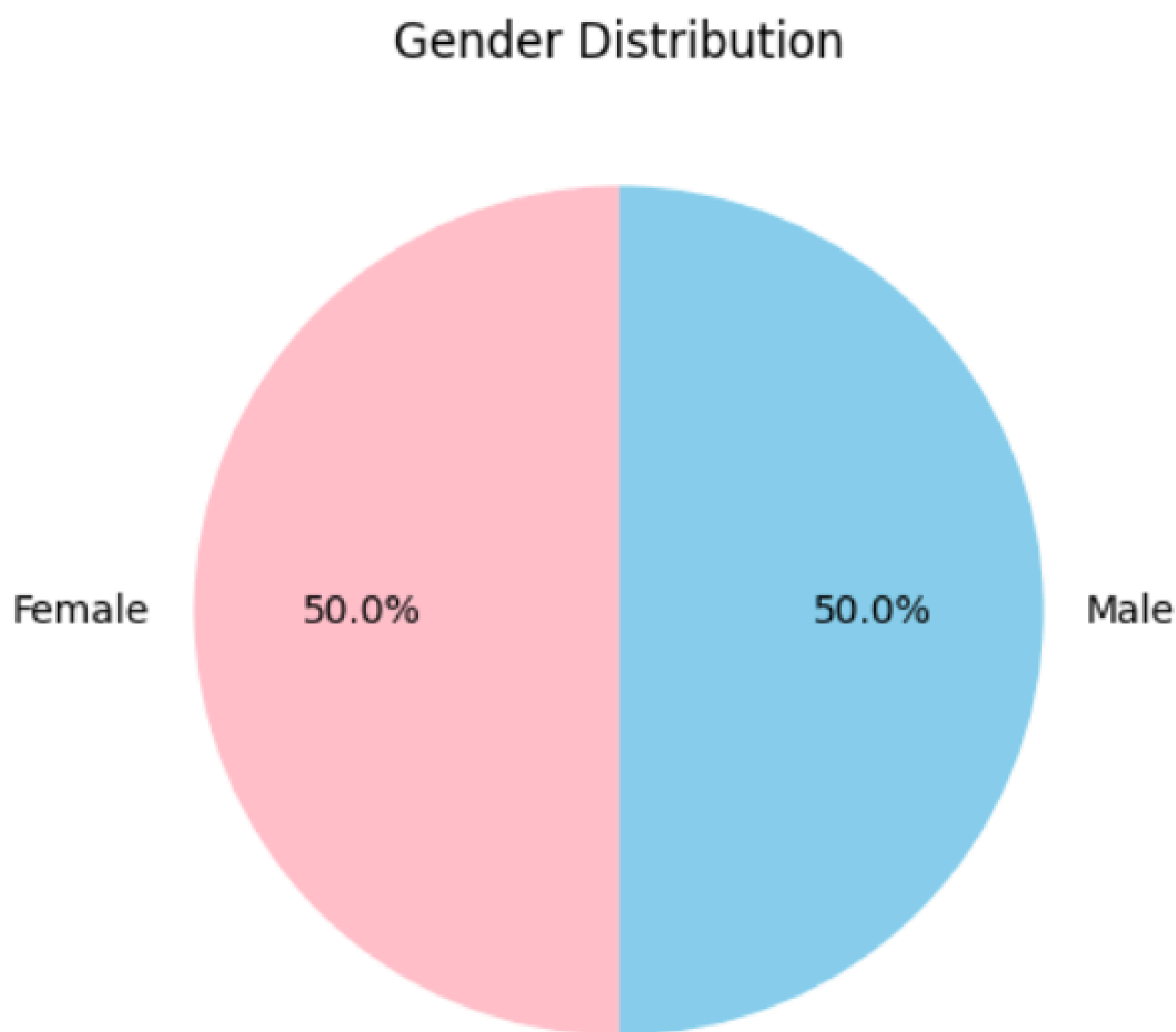
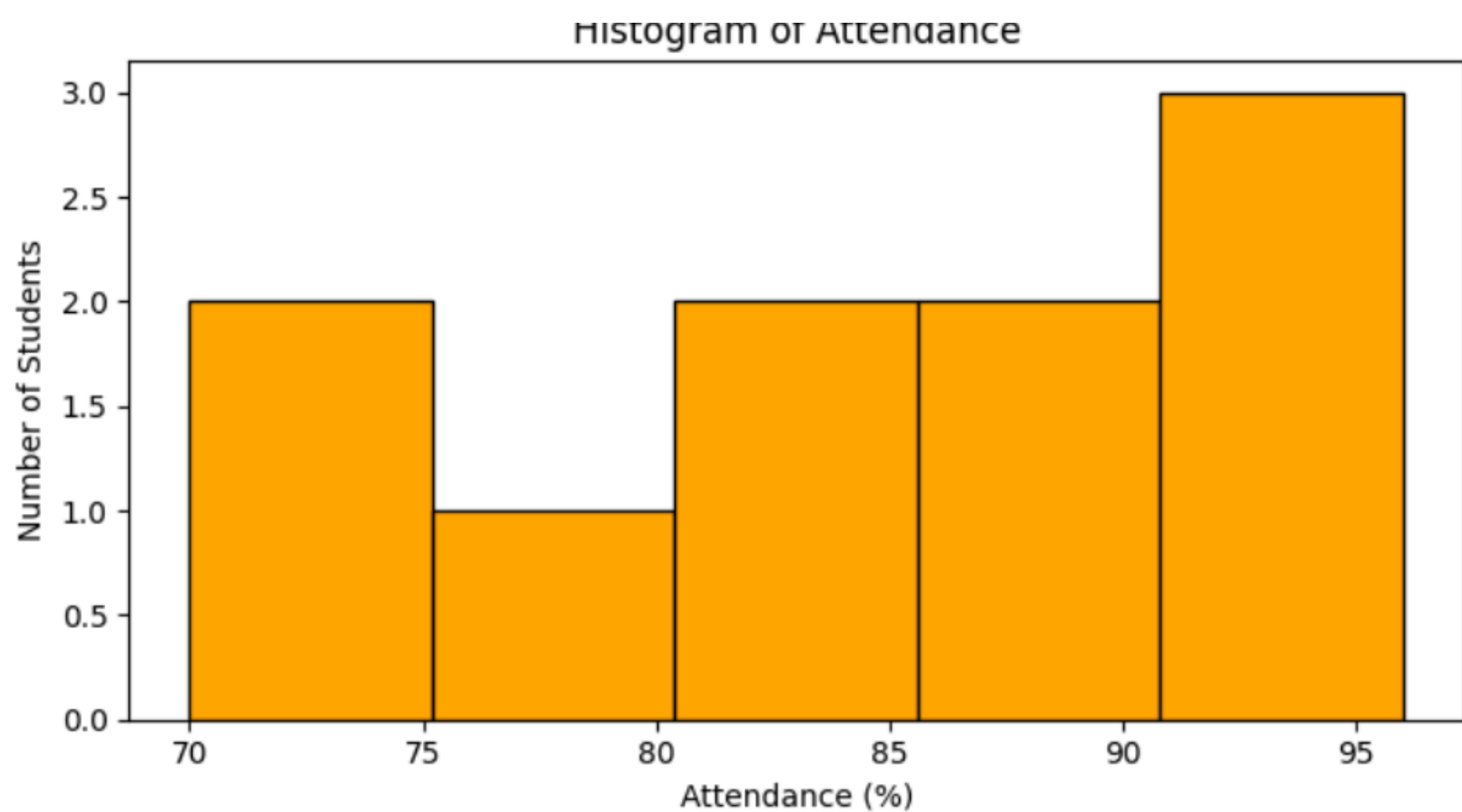
```

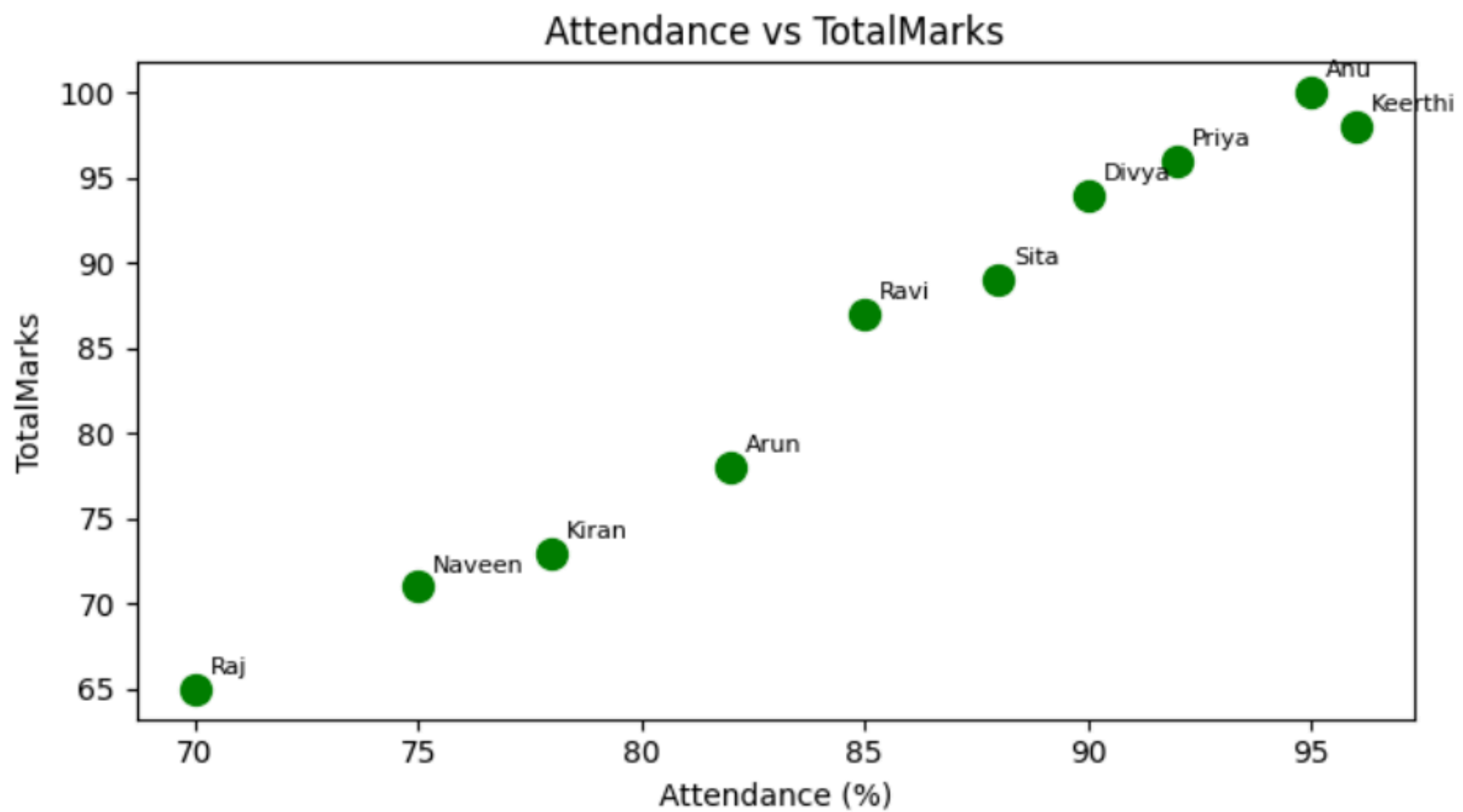
plt.figure(figsize=(7, 4))
plt.scatter(df['Attendance'], df['TotalMarks'], color='green', s=100)
for i, row in df.iterrows():
    plt.annotate(row['Name'], (row['Attendance'], row['TotalMarks']),
                 textcoords="offset points", xytext=(5, 5), fontsize=8)
plt.title('Attendance vs TotalMarks')
plt.xlabel('Attendance (%)')
plt.ylabel('TotalMarks')
plt.tight_layout()
plt.show()

```

OUTPUT:







Task H: Observations

- Write any three observations based on the analysis performed.
- State whether higher attendance appears to be associated with better academic performance.

OBSERVATIONS:

1. Students who attended more classes generally scored better marks.
For example, Anu had 95% attendance and scored the highest marks, while Raj had only 70% attendance and scored the lowest.
2. Female students performed slightly better than male students on average in both attendance and total marks.
3. Most students scored between 70 to 100 marks out of 120, which shows the overall class performance was average to good.

CONCLUSION:

Yes, higher attendance is linked to better performance.

Students with more than 90% attendance scored above the class average, and students with low attendance scored lower marks.

